

Project Report

Author: Ethan Wimmer

Date: 12/15/2025

Class: M491, Fall 2025, Linear Algebra for Data Science and Engineering

Linear Algebra Applications in Digital Color Analysis

Background

Color perception begins with the eye's response to light of different wavelengths. Our visual system has three types of cone cells where each type of cone is either sensitive to long, medium, or short wavelengths. Long wavelengths tend to be seen as red, medium as green, and short as blue. Because the brain interprets color based on the combined stimulation of these cones, nearly any perceivable color can be represented as a mixture of these three components. This biological phenomenon is the foundation of modern color models and is the direct motivation of mathematical representations of color in digital systems.

One of the most significant models of understanding color vision is the CIE 1931 color space. This was developed from experiments in which observers matched test lights using controlled mixtures of primary colors. The resulting functions, created from matching the colors, made the first standardized model of color vision that was compatible across devices. This in turn defined every visible color through three values in the XYZ coordinate system. This XYZ space forms a continuous model that includes all colors perceivable through our eyes.

Modern digital displays like computer monitors, televisions, and smartphone screens use a specific subset of this space depending on the device. This referred section is known as RGB color space. Standard RGB (sRGB) is mathematically derived from the CIE 1931 system through a linear transformation. The sRGB primaries (red, green, and blue) correspond to specific chromaticity coordinates from a two-dimensional slice of the CIE 1931 space shown below.

- $R = (0.64, 0.33)$
- $G = (0.30, 0.60)$
- $B = (0.15, 0.06)$

These primaries define the triangular region of colors a digital display can reproduce. Each pixel on a screen is made up of a red, green, and blue subpixel. By varying the intensity of each subpixel, the device creates the illusion of a single-colored pixel. This process relies entirely on numerical encoding where each subpixel intensity is stored as a value typically ranging from 0 to 255. This seemingly random range of values is because each color is stored as a byte of

information. One byte is made up of 8 binary bits creating 256 different values (0 to 255). Thus, a full pixel is represented by a triplet of bytes, making every digital color a structured set of numbers.

Mathematically, these three numerical components naturally form a vector in a three-dimensional space.

$$\mathbf{c} = \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Using operations like vector addition, scalar multiplication, dot products, and matrix transformations will allow for manipulation in digital imagery. These techniques can accomplish effects such as adjusting brightness, converting between color spaces, applying filters, and measuring similarity. Because images are simply large matrices of these RGB vectors, linear algebra provides a mathematical framework for understanding how devices store, modify, and display color.

In this way, visual perception, standardized color science, and modern digital image processing all intersect through linear algebra. This project builds on these ideas by exploring how color vectors and linear transformations can be used to analyze and estimate RGB values for colors in the real world.

Mathematical Background

The Structure of the RGB Color Space

Knowing that there are 256 possible intensity levels for three independent color channels (red, green, and blue), then the total number of possible colors that can be represented in the RGB space is:

$$256^3 = 16,777,216$$

These colors fill a three-dimensional space known as the RGB color cube, where the x-axis corresponds to red intensity, the y-axis corresponds to green intensity, and the z-axis corresponds to blue intensity. Therefore, every RGB color is a point in this cube. The diagonal from (0, 0, 0) to (255, 255, 255) represents grayscale values, while the corners correspond to pure primary or mixed secondary colors. Although expressed numerically rather than symbolically, RGB colors naturally form a vector space over the real numbers, enabling the use of linear algebra. This structure allows not only representation but also mathematical manipulation of colors.

Measuring Color Difference: Euclidean Distance

Because RGB defines a three-dimensional coordinate system, we can measure differences between colors using the standard Euclidean distance formula. For two colors:

$$\mathbf{c}_1 = (R_1, G_1, B_1), \mathbf{c}_2 = (R_2, G_2, B_2),$$

Their distance is:

$$d(\mathbf{c}_1, \mathbf{c}_2) = \sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}.$$

A smaller distance means that colors appear similar, a larger distance means that the colors differ noticeably, and zero means that the colors are the same. Although RGB distance does not perfectly match people's perception (other spaces like LAB improve this), it remains widely used because of its mathematical simplicity and computational efficiency.

Linear Transformations Between Color Spaces

Color models such as RGB and LMS are related by linear transformations. In matrix form,

$$\mathbf{c}_{\text{LMS}} = M\mathbf{c}_{\text{RGB}}$$

where M is a 3×3 matrix. Each row of M specifies how the RGB components contribute to one component of the new space. This transformation is linear because visual perception is approximately linear with respect to small intensity changes. The CIE XYZ system is designed so that mixing lights corresponds to vector addition, and increasing intensity corresponds to scalar multiplication. Therefore, color conversion is a matrix multiplication problem. For example, a simplified RGB to LMS conversion might use:

$$\begin{bmatrix} 0.3811 & 0.5783 & 0.0402 \\ 0.1967 & 0.7244 & 0.0782 \\ 0.0241 & 0.1288 & 0.8444 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

This would then be applied to every pixel, transforming the entire image.

Scalar Multiplication and Intensity Adjustments

Brightness adjustments use simple scalar multiplication:

$$\mathbf{C}_{\text{bright}} = s\mathbf{C}$$

where $s > 1$ increases brightness, $0 < s < 1$ decreases brightness, and $s = 1$ leaves color unchanged. Because RGB values must remain integers within $[0, 255]$, the result is typically rounded. Multiplying each channel by the same constant preserves the hue (the overall type of color), ratios between color channels, and direction of the vector in color space. Thus, the color becomes a brighter or darker version of itself without shifting toward another hue.

Tone and Temperature Shifts as Matrix Operations

Unlike brightness, warmth/coolness adjustments change channels unevenly. This requires matrix or vector multiplication. For example, a warm filter may look like this:

$$W = \begin{bmatrix} 1.10 & 0 & 0 \\ 0 & 1.00 & 0 \\ 0 & 0 & 0.90 \end{bmatrix}$$

where the red channel's intensity increases by 10%, the blue channel's intensity is reduced by 10%, and the green channel is left unchanged. Likewise, a cool filter may look like this:

$$C = \begin{bmatrix} 0.90 & 0 & 0 \\ 0 & 1.00 & 0 \\ 0 & 0 & 1.10 \end{bmatrix}$$

These matrices adjust the temperature of the color which allows for predictable color manipulation, and consistent behavior across all pixels.

Grayscale as a Weighted Linear Combination

Grayscale conversion reduces a three-dimensional color to a single brightness value. The standard ITU-R BT.601 formula uses:

$$Y = 0.299R + 0.587G + 0.114B$$

Mathematically, this is a dot product:

$$Y = [0.299 \quad 0.587 \quad 0.114] \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

These weights are meant to reflect the sensitivity of light to our eyes. Thus, red contributes about 30% to perceived brightness, green contributes about 59%, and blue contributes around 11%. In short, grayscale conversion is a linear projection from RGB space onto a brightness axis.

Gaussian Blur and Convolution Kernels

Gaussian blur is based on the Gaussian distribution. This distribution is a smooth bell-shaped function that assigns larger weights to pixels closer to the center, and smaller weights to pixels farther away. This is used in blurring techniques so that nearby pixels influence the pixel's value more strongly than distant ones. The two-dimensional Gaussian function is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

where x and y are spatial offsets from the center pixel, and σ is the standard deviation that controls the strength of blur. The exponential term ensures weights decrease smoothly with distance, and the normalization factor $\frac{1}{2\pi\sigma^2}$ ensures the total area under the curve equals 1. This function is radially symmetric, meaning the blur is the same in all directions. Digital images are discrete, so the continuous Gaussian must be sampled at integer points. For a kernel of size $n \times n$, we evaluate $G(x, y)$ for integer values of x and y centered around zero:

$$x, y \in \left\{-\frac{n-1}{2}, \dots, 0, \dots, \frac{n-1}{2}\right\}$$

These sampled values form a Gaussian kernel. Because sampling introduces rounding errors, the kernel is normalized so that:

$$\sum_{i=1}^n \sum_{j=1}^n K_{ij} = 1.$$

This normalization preserves the overall brightness of the image. For example, a commonly used 5×5 Gaussian kernel is:

$$K = \frac{1}{256} \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \\ 4 & 16 & 24 & 16 & 4 \\ 6 & 24 & 36 & 24 & 6 \\ 4 & 16 & 24 & 16 & 4 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}.$$

As expected, the largest value is at the center, and values decrease symmetrically as the distance from the center increases. For each pixel, the kernel is placed over a matching $n \times n$ patch of the image. The new pixel value is computed as:

$$\text{Blurred Pixel} = \sum_{i=1}^n \sum_{j=1}^n K_{ij} \cdot P_{ij},$$

where P_{ij} are the pixel intensities under the kernel. This operation is equivalent to a dot product between the flattened kernel vector, and the flattened image patch vector. Seeing that dot products and summation are linear, convolution is a linear transformation. Applying this process across all pixels and each color channel produces the final blurred image.

One problem that arises with image blurring is that at image boundaries, the kernel may extend beyond the available pixels. Some common techniques to resolve this problem include zero padding, edge replication, and reflective padding. For this project, reflective padding was used. This duplicates kernel values that are outside the boundary of the image to prevent errors and maintain a smooth edge.

Gaussian blur demonstrates several core linear algebra concepts: kernels as matrices, convolution as repeated dot products, linearity and superposition, and transformation of image data represented as large matrices. This makes Gaussian blur a powerful example of how abstract linear algebra concepts directly enable practical image processing techniques.

Results and Calculations

The goal of the project was to take any given color and approximate its RGB values manually or automatically using linear algebra operations. Currently, only manual implementations have been accomplished. For the structure of this experiment, the user is given a random color and must use tools to get another color as close to the given color as possible. The user can only see the RGB value of the color being modified as it updates, not the random color.

Once the user believes that the modifiable color now has the same value as the given color, then the RGB value of the test color can be revealed. Once this happens, the Euclidean distance between the two colors is noted and the process is repeated with a new color. For each iteration, the user starts with a perfectly grey square (127, 127, 127), and a test square taken from a dataset of real-world colors.

Dataset

A dataset of over 1,500 Sherwin Williams paint colors was used. Each entry includes the name of the color, a visual of the color, and the RGB value of the color. The dataset provided reference colors for evaluation and randomized testing.

Attempt 1: Manual Color matching (Additive Buttons)

For the first implementation, only the buttons “More” and “Less” were created for each color channel. Each time a button for a specific color channel was pressed, like “More Red” or “Less Red”, the value of that primary would change by half of the divisibility of the currently value. For example, an increase from the value 127 would then become 191. This is because 127 is halfway to 255, so the intensity of the color increases by one fourth (64). This was done to create an efficient and easy way of comparing the two colors; however, this was not the case. Pressing a button would seem to change the value randomly, which in turn created a very frustrating design for matching colors. For results, the typical distance between the two RGB values was around 11.

Attempt 2: Scalar/Matrix Based Adjustments

For the second interface, the buttons “Lighter”, “Darker”, “Warmer”, and “Cooler” were added. Additionally, the previous buttons were fixed so that each “More” or “Less” button would increment by a constant value of 5. For the “Lighter” and “Darker” buttons, a scaler of 1.1 would be applied to the RGB vector to make the color lighter, and a value of 0.9 to make the color darker. Similarly, the “Warmer” and “Cooler” buttons worked through matrix multiplication with weighted red and blue values. A warmer color would have more red and less blue, and a cooler color would have less red and more blue. This produced much better results, having an average error reduced to a distance of around 5 units, and more predictable/efficient adjustments.

Attempt 3: Color Slider

For the last implementation, the “More” and “Less” buttons were replaced by a slider tool, and everything else was kept the same. This allowed the user to slide each RGB value very quickly, and very accurately. This means that the user can precisely select values while preventing complex, inconsistent controls. Likewise, the buttons from the previous attempt were kept ensuring that the user could change values based on common color terminology. For results, the typical distance was between 1 and 2 units from the exact RGB color. This ended as the last and best performing implementation for the project.

Code and Outputs

Imports and Data Cleaning

```
import ipywidgets as widgets
import io
import tabula
import pandas as pd
import numpy as np
import random
import math
import tkinter as tk
import plotly.graph_objects as go
from IPython.display import display, clear_output
from PIL import Image, ImageDraw
from IPython.display import Image as IPyImage
from scipy.signal import convolve2d

tabula.convert_into("sherwin_colors.pdf", "sherwin_colors.csv", output_format="csv", pages="all", stream=True)
df = pd.read_csv("sherwin_colors.csv", skiprows=[95,96,97,99,100,101,106,107,108,119,120,121])
df.columns = ["Num", "Name", "Loc", "Red", "Green", "Blue", "Hex", "Color"]

new_rows = [
    {"Num": "SW2807", "Name": "Rookwood Medium Brown", "Loc": None, "Red": 110, "Green": 82, "Blue": 65, "Hex": "6E5241", "Color": None},
    {"Num": "SW2809", "Name": "Rookwood Shutter Green", "Loc": None, "Red": 48, "Green": 59, "Blue": 57, "Hex": "303B39", "Color": None},
    {"Num": "SW2814", "Name": "Rookwood Antique Gold", "Loc": None, "Red": 165, "Green": 130, "Blue": 88, "Hex": "A58258", "Color": None},
    {"Num": "SW2826", "Name": "Colonial Revival Green Stone", "Loc": None, "Red": 163, "Green": 155, "Blue": 126, "Hex": "A39B7E", "Color": None}
]
df = pd.concat([df, pd.DataFrame(new_rows)], ignore_index=True)

for i in range(len(df)):
    df.loc[i, "Color"] = Image.new('RGB', (100, 100), color=(df["Red"][i], df["Green"][i], df["Blue"][i]))
```

	Num	Name	Loc	Red	Green	Blue	Hex	Color
0	SW0001	Mulberry Silk	NaN	148	118	108	94766C	<PIL.Image.Image image mode=RGB size=100x100 a...
1	SW0002	Chelsea Mauve	NaN	190	172	159	BEAC9F	<PIL.Image.Image image mode=RGB size=100x100 a...
2	SW0003	Cabbage Rose	NaN	197	159	145	C59F91	<PIL.Image.Image image mode=RGB size=100x100 a...
3	SW0004	Rose Brocade	NaN	153	108	110	996C6E	<PIL.Image.Image image mode=RGB size=100x100 a...
4	SW0005	Deepest Mauve	NaN	109	89	90	6D595A	<PIL.Image.Image image mode=RGB size=100x100 a...

Random Color Selector

```
x = random.randint(0, len(df))

target = df["Color"][x]
target_rgb = (int(df["Red"][x]), int(df["Green"][x]), int(df["Blue"][x]))

print(df["Name"][x])
display(target)
```

Jalapeño



Grayscale Conversion

```
img = Image.open("bird_image.jpg")
rgb = np.array(img).astype(float)

weights = np.array([0.299, 0.587, 0.114])
gray = rgb.dot(weights)

gray_img = Image.fromarray(gray.astype(np.uint8))
display(gray_img)
```



3D Visualization

```
color = target_rgb
R, G, B = color
cube_points = np.array([
    [0, 0, 0], [255, 0, 0], [0, 255, 0], [0, 0, 255],
    [255, 255, 0], [255, 0, 255], [0, 255, 255], [255, 255, 255]
])

fig = go.Figure()

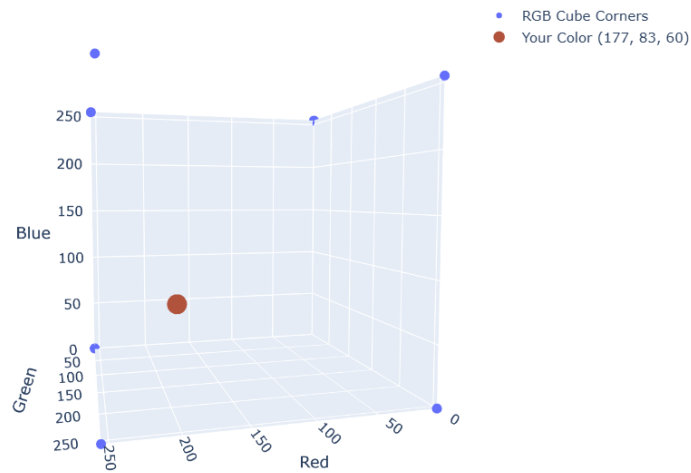
fig.add_trace(go.Scatter3d(
    x=cube_points[:,0], y=cube_points[:,1], z=cube_points[:,2],
    mode='markers',
    marker=dict(size=5),
    name='RGB Cube Corners'
))

fig.add_trace(go.Scatter3d(
    x=[R], y=[G], z=[B],
    mode='markers',
    marker=dict(size=10, color=f'rgb({R},{G},{B})'),
    name=f'Your Color {color}'
))

fig.update_layout(
    scene=dict(
        xaxis=dict(title='Red', range=[0,255]),
        yaxis=dict(title='Green', range=[0,255]),
        zaxis=dict(title='Blue', range=[0,255])
    ),
    width=700,
    height=600,
    title="Interactive 3D RGB Color Plot"
)

fig.show()
```

Interactive 3D RGB Color Plot



Gaussian Blur

```
def gaussian_kernel(size, sigma):
    ax = np.linspace(-(size // 2), size // 2, size)
    xx, yy = np.meshgrid(ax, ax)

    kernel = np.exp(-(xx**2 + yy**2) / (2.0 * sigma**2))
    kernel = kernel / np.sum(kernel)
    return kernel

def convolve(image, kernel):
    img_h, img_w = image.shape
    k = kernel.shape[0]
    pad = k // 2

    padded = np.pad(image, pad, mode='edge')

    output = np.zeros_like(image, dtype=float)

    for i in range(img_h):
        for j in range(img_w):
            region = padded[i:i+k, j:j+k]
            output[i, j] = np.sum(region * kernel)
    return output

def gaussian_blur_manual(img_path, kernel_size, sigma):
    img = Image.open(img_path).convert("RGB")
    img_np = np.array(img, dtype=float)

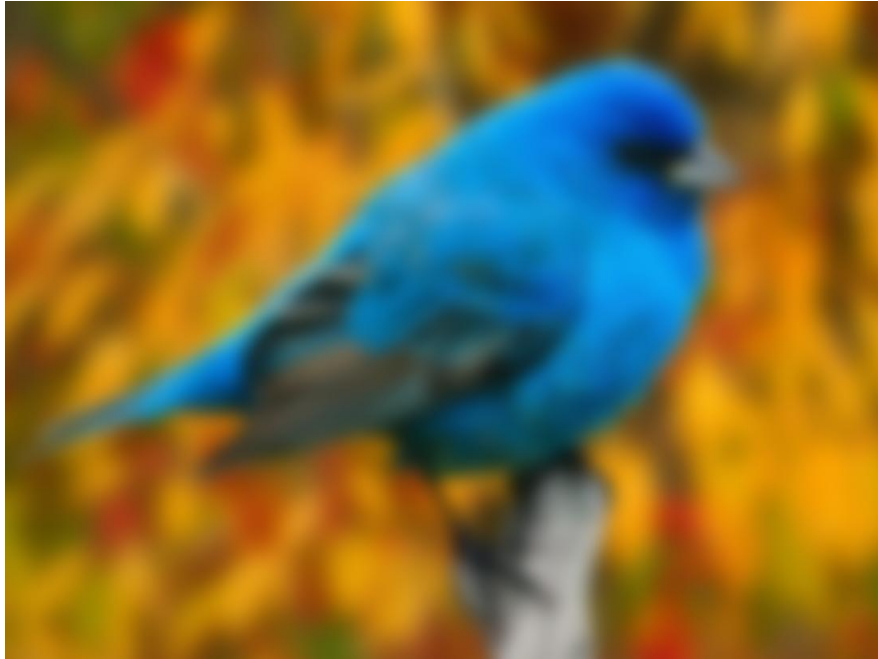
    kernel = gaussian_kernel(kernel_size, sigma)

    blurred = np.zeros_like(img_np)

    for c in range(3):
        blurred[:, :, c] = convolve(img_np[:, :, c], kernel)

    blurred = np.clip(blurred, 0, 255).astype(np.uint8)
    return Image.fromarray(blurred)

blurred_img = gaussian_blur_manual("bird_image.jpg", kernel_size=101, sigma=17)
display(blurred_img)
```



Manual Implementation 1

```
color = {'R': 127, 'G': 127, 'B': 127}

color_out = widgets.Output()
target_out = widgets.Output()
rgb_out = widgets.Output()
target_rgb_out = widgets.Output()

with target_out:
    img_byte_arr = io.BytesIO()
    target.save(img_byte_arr, format='PNG')
    display(IPyImage(img_byte_arr.getvalue()))

def show_color():
    with color_out:
        clear_output(wait=True)
        img = Image.new('RGB', (100, 100), (color['R'], color['G'], color['B']))
        with io.BytesIO() as output:
            img.save(output, format="PNG")
            data = output.getvalue()
            display(IPyImage(data))

    with rgb_out:
        clear_output(wait=True)
        print(f"RGB: ({color['R']}, {color['G']}, {color['B']})")

def check_color(b):
    with target_rgb_out:
        clear_output(wait=True)
        print(f"Target RGB: {target_rgb}")
```

```

def color_quant(value):
    dec = len(str((value+1)/256))-3
    quant = 64/(2**dec)
    quant = math.ceil(quant)
    return quant

def handle_click(option):
    if option == "More Red":
        color['R'] = min(color['R'] + color_quant(color['R']), 255)
    elif option == "Less Red":
        color['R'] = max(color['R'] - color_quant(color['R']), 0)
    elif option == "More Green":
        color['G'] = min(color['G'] + color_quant(color['G']), 255)
    elif option == "Less Green":
        color['G'] = max(color['G'] - color_quant(color['G']), 0)
    elif option == "More Blue":
        color['B'] = min(color['B'] + color_quant(color['B']), 255)
    elif option == "Less Blue":
        color['B'] = max(color['B'] - color_quant(color['B']), 0)
    elif option == "Check Color":
        check_color(None)
    show_color()

button_labels = ["More Red", "Less Red", "More Green", "Less Green", "More Blue", "Less Blue", "Check Color"]
buttons = [widgets.Button(description=label) for label in button_labels]

for btn, label in zip(buttons, button_labels):
    btn.on_click(lambda b, lbl=label: handle_click(lbl))

button_box = widgets.HBox(buttons)

display(widgets.HBox([widgets.VBox([target_out, target_rgb_out]), widgets.VBox([color_out, rgb_out])]))
display(button_box)

show_color()

```



RGB: (127, 127, 127)

More Red	Less Red	More Green	Less Green	More Blue	Less Blue	Check Color
----------	----------	------------	------------	-----------	-----------	-------------

Manual Implementation 2

```
color = {'R': 127, 'G': 127, 'B': 127}

color_out = widgets.Output()
target_out = widgets.Output()
rgb_out = widgets.Output()
target_rgb_out = widgets.Output()

with target_out:
    img_byte_arr = io.BytesIO()
    target.save(img_byte_arr, format='PNG')
    display(IPyImage(img_byte_arr.getvalue()))

def show_color():
    with color_out:
        clear_output(wait=True)
        img = Image.new('RGB', (100, 100), (color['R'], color['G'], color['B']))
        with io.BytesIO() as output:
            img.save(output, format="PNG")
            data = output.getvalue()
        display(IPyImage(data))

    with rgb_out:
        clear_output(wait=True)
        print(f"RGB: ({color['R']}, {color['G']}, {color['B']})")

def check_color(b):
    with target_rgb_out:
        clear_output(wait=True)
        print(f"Target RGB: {target_rgb}")

def color_quant(value):
    dec = len(str((value+1)/256))-3
    quant = 64/(2**dec)
    quant = math.ceil(quant)
    return quant
```

```

def handle_click(option):
    if option == "More Red":
        color['R'] = min(color['R'] + 5, 255)
    elif option == "Less Red":
        color['R'] = max(color['R'] - 5, 0)
    elif option == "More Green":
        color['G'] = min(color['G'] + 5, 255)
    elif option == "Less Green":
        color['G'] = max(color['G'] - 5, 0)
    elif option == "More Blue":
        color['B'] = min(color['B'] + 5, 255)
    elif option == "Less Blue":
        color['B'] = max(color['B'] - 5, 0)
    elif option == "Check Color":
        check_color(None)
    elif option == "Lighter":
        color['R'] = round(min(1.1*color['R'], 255))
        color['G'] = round(min(1.1*color['G'], 255))
        color['B'] = round(min(1.1*color['B'], 255))
    elif option == "Darker":
        color['R'] = round(max(0.9*color['R'], 0))
        color['G'] = round(max(0.9*color['G'], 0))
        color['B'] = round(max(0.9*color['B'], 0))
    elif option == "Warmer":
        color['R'] = round(min(1.1*color['R'], 255))
        color['B'] = round(max(0.9*color['B'], 0))
    elif option == "Cooler":
        color['R'] = round(max(0.9*color['R'], 0))
        color['B'] = round(min(1.1*color['B'], 255))
    show_color()

button_labels = ["More Red", "Less Red", "More Green", "Less Green", "More Blue", "Less Blue", "Lighter", "Darker", "Warmer", "Cooler", "Check Color"]
buttons = [widgets.Button(description=label) for label in button_labels]

for btn, label in zip(buttons, button_labels):
    btn.on_click(lambda b, lbl=label: handle_click(lbl))

button_box = widgets.HBox(buttons)

display(widgets.HBox([widgets.VBox([target_out, target_rgb_out]), widgets.VBox([color_out, rgb_out])]))
display(button_box)

show_color()

```



RGB: (127, 127, 127)

More Red	Less Red	More Green	Less Green	More Blue	Less Blue	Lighter	Darker	Warmer	Cooler	Check Color
----------	----------	------------	------------	-----------	-----------	---------	--------	--------	--------	-------------

Manual Implementation 3

```
color = {'R': 127, 'G': 127, 'B': 127}

color_out = widgets.Output()
target_out = widgets.Output()
rgb_out = widgets.Output()
target_rgb_out = widgets.Output()

with target_out:
    img_byte_arr = io.BytesIO()
    target.save(img_byte_arr, format='PNG')
    display(IPyImage(img_byte_arr.getvalue()))

def show_color():
    with color_out:
        clear_output(wait=True)
        img = Image.new('RGB', (100, 100), (color['R'], color['G'], color['B']))
        with io.BytesIO() as output:
            img.save(output, format="PNG")
            data = output.getvalue()
        display(IPyImage(data))

    with rgb_out:
        clear_output(wait=True)
        print(f"RGB: ({color['R']}, {color['G']}, {color['B']})")

def check_color():
    with target_rgb_out:
        clear_output(wait=True)
        print(f"Target RGB: {target_rgb}")

def handle_click(option):
    if option == "Lighter":
        color['R'] = round(min(1.1*color['R'], 255))
        color['G'] = round(min(1.1*color['G'], 255))
        color['B'] = round(min(1.1*color['B'], 255))
    elif option == "Darker":
        color['R'] = round(max(0.9*color['R'], 0))
        color['G'] = round(max(0.9*color['G'], 0))
        color['B'] = round(max(0.9*color['B'], 0))
    elif option == "Warmer":
        color['R'] = round(min(1.1*color['R'], 255))
        color['B'] = round(max(0.9*color['B'], 0))
    elif option == "Cooler":
        color['R'] = round(max(0.9*color['R'], 0))
        color['B'] = round(min(1.1*color['B'], 255))
    elif option == "Check Color":
        check_color()
    show_color()
    slider_R.value = color['R']
    slider_G.value = color['G']
    slider_B.value = color['B']
```

```

def on_slider_change(change, channel):
    color[channel] = change['new']
    show_color()

button_labels = ["Lighter", "Darker", "Warmer", "Cooler", "Check Color"]
buttons = [widgets.Button(description=label) for label in button_labels]

slider_R = widgets.IntSlider(description='Red', min=0, max=255, value=color['R'])
slider_G = widgets.IntSlider(description='Green', min=0, max=255, value=color['G'])
slider_B = widgets.IntSlider(description='Blue', min=0, max=255, value=color['B'])

slider_R.observe(lambda change: on_slider_change(change, 'R'), names='value')
slider_G.observe(lambda change: on_slider_change(change, 'G'), names='value')
slider_B.observe(lambda change: on_slider_change(change, 'B'), names='value')

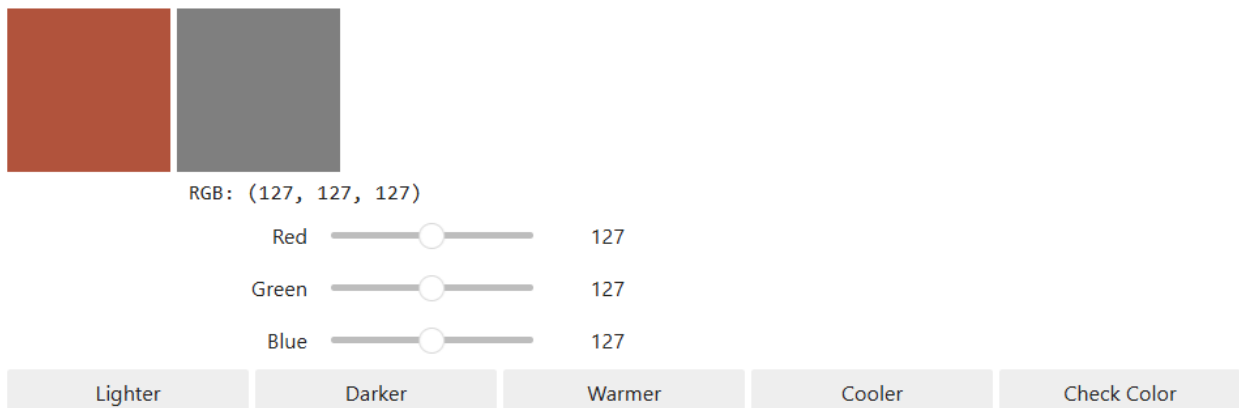
for btn, label in zip(buttons, button_labels):
    btn.on_click(lambda b, lbl=label: handle_click(lbl))

slider_box = widgets.VBox([slider_R, slider_G, slider_B])
button_box = widgets.HBox(buttons)

display(widgets.HBox([widgets.VBox([target_out, target_rgb_out]), widgets.VBox([color_out, rgb_out, slider_box])]))
display(button_box)

show_color()

```



Conclusions

This project examined how linear algebra underlies the representation and analysis of color in digital imaging. Beginning with color perception and the CIE 1931 color space, modern RGB systems are derived from perceptual models through linear transformations. By showing how digital images store color information numerically using RGB values as bytes, color data naturally relates to mathematical interpretation as vectors in a three-dimensional space.

Through the mathematical background, key linear algebra concepts were applied directly to color processing tasks. Euclidean distance provided a quantitative measure of color similarity, while scalar multiplication and matrix transformations enabled brightness and color temperature adjustments. Grayscale conversion was shown to be a weighted linear projection, and Gaussian

blur was displayed as a convolution process built from dot products and matrix operations. Together, these illustrate how abstract linear algebra concepts can be transformed into practical image processing techniques.

The experimental portion of the project brought some of these ideas to light with hands-on implementations. Initial additive color adjustments proved inefficient and unpredictable, highlighting the limitations of a first approach. On the other hand, scalar and matrix multiplication proved to significantly improved accuracy and control. The final interface added a slider, which achieved near-perfect color matching. This typically produced a distance within one to two units of the target color, demonstrating the effectiveness of linear methods for precise color estimation.

Overall, this project showed that linear algebra is not only a theoretical tool but also a foundational framework for modern digital imaging. Future work could extend these methods to automated solvers, perceptually uniform color spaces, or full-image processing applications. Such extensions would likely show the power of linear algebra in bridging human perception, to mathematical structure.

References

- [1] C.P. Bruter, *Mathematics and Art*, Springer, Paris (2002).
- [2] S. Carstensen, E.P. Stephan, *Adaptive boundary-element methods for transmission problems*, Journal of Mathematical Analysis and Applications, 106, 367–413 (1985).
- [3] S. Dostoglou, *On the asymptotics of the finite energy solutions of the Yang-Mills-Higgs equations*, Journal of Mathematical Physics, 31, 2490–2496 (1990).
- [4] N. Lesmoir-Gordon, M. Frame, B. Mandelbrot, N. Neger, *Mandelbrot's World of Fractals*, Key Curriculum Press (2005).
- [5] N.J. Higham, *Color Spaces and Digital Imaging*, SIAM Review, 65(1), 3–19 (2023).
- [6] A.R. Smith, *Color Gamut Transform Pairs*, SIGGRAPH Conference Proceedings (1978).
- [7] R.C. Gonzalez, R.E. Woods, *Digital Image Processing*, 4th Edition, Pearson, New York (2018).
- [8] M.D. Fairchild, *Color Appearance Models*, 3rd Edition, Wiley-IS&T, Chichester, UK (2013).
- [9] G. Wyszecki, W.S. Stiles, *Color Science: Concepts and Methods, Quantitative Data and Formulae*, 2nd Edition, Wiley, New York (1982).
- [10] G. Sharma (Ed.), *Digital Color Imaging Handbook*, CRC Press, Boca Raton, FL (2002).
- [11] C.A. Poynton, *Digital Video and HDTV — Algorithms and Interfaces*, Morgan Kaufmann / Elsevier, Burlington, MA (2012).
- [12] G. Sharma, W. Wu, E.N. Dalal, *The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations*, Color Research & Application, 30, 21–30 (2005).
- [13] W.D. Wright, *A re-determination of the trichromatic coefficients of the spectral colours*, Transactions of the Optical Society, 30, 141–164 (1929).
- [14] R.W.G. Hunt, *The Reproduction of Colour*, 6th Edition, Wiley-IS&T, Hoboken, NJ (2004).
- [15] CIE (Commission Internationale de l'Éclairage), *Colorimetry*, CIE Publication No. 15.2, Central Bureau of the CIE, Vienna (2004).
- [16] G.P. Michon, *Final answers: Perimeter of an ellipse*, Numericana <http://numericana.com/answer/ellipse.htm> (updated May 17, 2011).
- [17] Pillow Documentation, <https://pillow.readthedocs.io>
- [18] NumPy Documentation, <https://numpy.org/docs>
- [19] Matplotlib 3D Plotting Gallery, <https://matplotlib.org/stable/gallery/mplot3d>